

Experimental Evaluation and Optimization of Real-Time Scheduling on STM32

Vladimir Toporkov, Artem Burmyakov
Innopolis University

I. INTRODUCTION

A. Contribution

Our contributions are about evaluation of schedulers for STM32 microcontroller:

- we first implement Super Loop scheduler,
- then we implement chunked Earliest Deadline First (EDF) scheduler, and finally
- we evaluate their performance on a real STM32 hardware.

II. BACKGROUND ON STM32

The experimental platform consists of a uniprocessor STM32F767 microcontroller and external sensors connected through its hardware peripherals, as shown in Fig. 1.

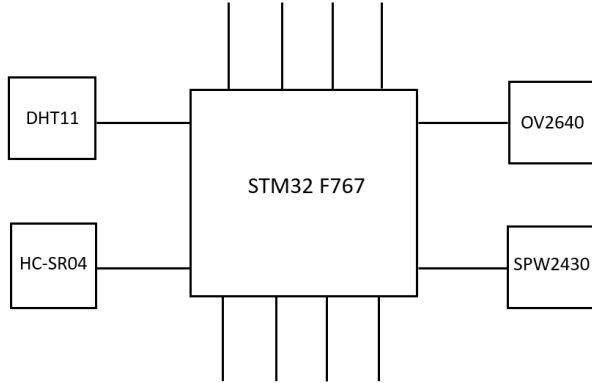


Fig. 1. STM32 hardware platform and connected sensors.

The hardware components are presented in Table I.

TABLE I
EXPERIMENTAL HARDWARE PLATFORM.

Component	Purpose
STM32F767	Scheduling and task execution
DHT11	Temperature and humidity measurement
HC-SR04	Distance measurement
OV2640	Image capturing
SPW2430	Acoustic signal acquisition

III. SYSTEM MODEL

We model the workload by a set of periodic tasks $\mathcal{T} = \{\tau_1, \dots, \tau_n\}$ executed on a uniprocessor. We consider only implicit deadlines, i.e., $D_i = T_i$. Each task $\tau_i = (C_i, T_i = D_i)$ generates a sequence of jobs J_i^j , where j denotes the scheduled

release index. Consecutive scheduled releases satisfy $R_i^{j+1} = R_i^j + T_i$.

TABLE II
SYSTEM MODEL NOTATION.

Symbol	Meaning
$\mathcal{T}, n$	Task set and number of tasks
τ_i	Task i
C_i	Execution time of a job of τ_i
T_i	Task period
$D_i = T_i$	Relative deadline, equal to the period
J_i^j	Job associated with the j -th scheduled release of τ_i
R_i^j	Scheduled release time
f_i^j	Completion time of an executed job
$d_i^j = R_i^j + D_i$	Absolute deadline
$U = \sum_{i=1}^n C_i/T_i$	Task-set utilization

Table II summarizes the notation. Release indices include skipped activations, for which no completion time is defined.

- A schedule specifies which job executes on the processor at each time instant, or whether the processor executes no job.
- Preemption interrupts a job before completion to execute another job. The interrupted job may resume later.
- A completed job J_i^j has missed its deadline if $f_i^j > d_i^j$.

IV. SCHEDULING PRINCIPLES

A. Super Loop Scheduler

The considered Super Loop scheduler repeatedly scans tasks in a fixed order and executes each selected job non-preemptively. To illustrate its behavior, let us consider the two periodic tasks in Table III. Both tasks first release at $t = 0$.

TABLE III
TASK SET USED IN THE SUPER LOOP EXAMPLE.

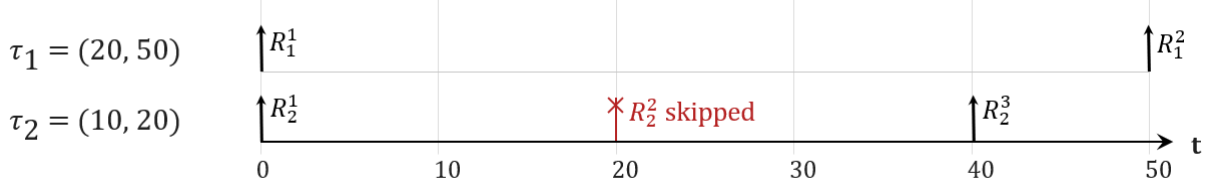
Task	C_i	T_i	D_i
τ_1	20	50	50
τ_2	10	20	20

The resulting Super Loop schedule is shown in Fig. 2.

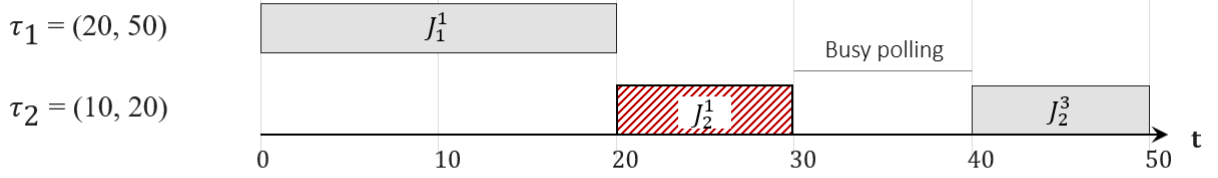
Let t^{cur} denote the current system time and t_i^{next} the next scheduled release time of task τ_i . During execution of J_i^j , the release variable t_i^{next} retains R_i^j . At completion, f_i^j denotes the completion time, and a deadline miss is recorded if $f_i^j > d_i^j$. The scheduler then advances t_i^{next} by T_i . While $t_i^{next} <$

$$\tau_i = (C_i, D_i = T_i)$$

Scheduled Releases



Super Loop execution scenario



Absolute Deadline: 20 Completion: 30

J_j^i – job execution R_j^i – scheduled release time

Fig. 2. Super Loop execution scenario for the task set in Table III.

f_i^j , the scheduled release at t_i^{next} is counted as skipped and t_i^{next} is advanced by T_i again. A release at $t_i^{next} = f_i^j$ is retained.

Algorithm 1 summarizes the illustrated scheduling logic. Since $R_i^1 = 0$, the release index is $j = 1 + t_i^{next}/T_i$, including skipped releases. The clock used by NOW starts at zero.

The corresponding scheduling state is shown in Table IV.

TABLE IV
SUPER LOOP SCHEDULING STATE FOR FIG. 2.

t^{cur}	t_1^{next}	t_2^{next}	Deadline miss check	Release skip check
0	0	0	–	–
20	50	0	$20 < R_1^1 + 50$	$20 < R_1^2$
30	50	40	$30 > R_2^1 + 20$	$30 > R_2^2$
40	50	40	–	–
50	50	60	$50 < R_2^3 + 20$	$50 < R_2^4$

In Fig. 2, J_1^1 executes from $t = 0$ to $t = 20$. Then, J_2^1 executes until $t = 30$, missing its deadline at $t = 20$. At completion, the release scheduled for $R_2^2 = 20$ is counted as skipped and t_2^{next} advances to $R_2^3 = 40$. Thus, no J_2^2 is executed.

B. Earliest Deadline First

Unlike Super Loop, Earliest Deadline First (EDF) selects jobs according to their absolute deadlines.

We first consider preemptive EDF with immediate preemption. At each scheduling decision, the ready job with the

Algorithm 1 Super Loop Scheduler

```

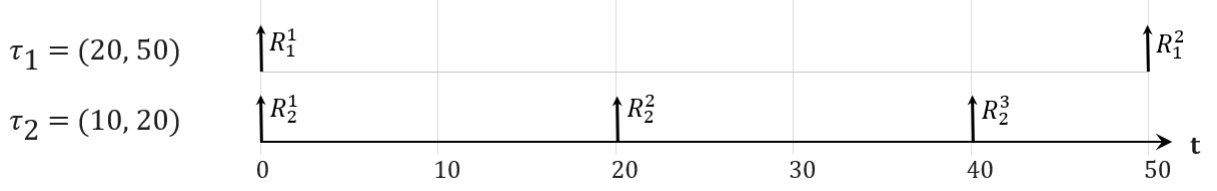
1: function SUPERLOOP( $\mathcal{T}$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:   end for
5:   while true do
6:     for  $i \leftarrow 1$  to  $n$  do ▷ Fixed task order
7:        $t^{cur} \leftarrow \text{NOW}$ 
8:       if  $t^{cur} \geq t_i^{next}$  then
9:          $j \leftarrow 1 + t_i^{next}/T_i$ 
10:         $R_i^j \leftarrow t_i^{next}$ 
11:        EXECUTE( $J_i^j$ ) ▷ Non-preemptive
12:         $f_i^j \leftarrow \text{NOW}$ 
13:        if  $f_i^j > R_i^j + D_i$  then
14:          RECORDMISS( $J_i^j$ )
15:        end if
16:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
17:        while  $t_i^{next} < f_i^j$  do
18:          RECORDSKIP( $\tau_i, t_i^{next}$ )
19:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
20:        end while
21:      end if
22:    end for
23:  end while
24: end function

```

earliest absolute deadline d_i^j is selected (Section III). If a newly released job has an earlier absolute deadline than the running

$$\tau_i = (C_i, D_i = T_i)$$

Scheduled Releases



EDF execution scenario

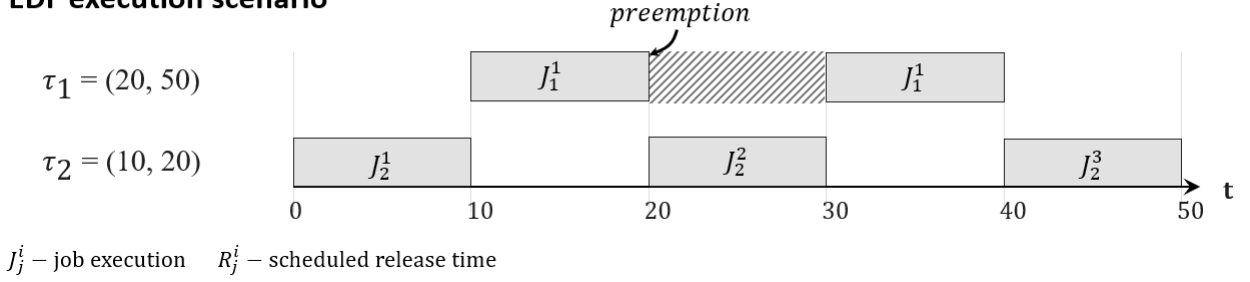


Fig. 3. EDF execution scenario for the task set in Table III. Hatching indicates a preempted job awaiting resumption.

job, it preempts the running job.

Fig. 3 illustrates EDF scheduling for the task set in Table III.

At $t = 20$, J_2^2 is released with $d_2^2 = 40 < d_1^1 = 50$ and preempts J_1^1 . After J_2^2 completes at $t = 30$, J_1^1 resumes and completes at $t = 40$.

The STM32 implementation approximates preemption using execution chunks of maximum duration Δ . Scheduling decisions are made only at chunk boundaries. Therefore, an earlier-deadline job released during a chunk may wait until the current chunk completes.

In Fig. 4, J_1^1 executes a chunk from $t = 18$ ms to $t = 22$ ms. Job J_2^2 is released at $t = 20$ ms, but the scheduler rechecks the ready jobs only at $t = 22$ ms. Since J_2^2 has the earlier absolute deadline, it is selected at the next chunk boundary.

The resulting preemption delay is $\delta = 2$ ms. In general, $0 \leq \delta < \Delta$.

Let $\mathcal{R}$ denote the set of ready jobs. Algorithm 2 summarizes

the implemented Chunked EDF logic, where job selection is repeated after each execution chunk.

V. PERFORMANCE EVALUATION

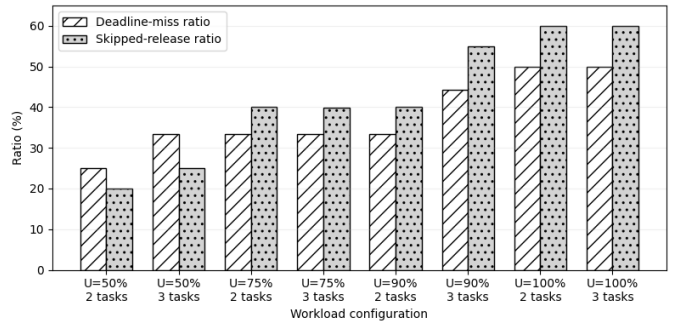


Fig. 5. Per-task deadline-miss and skipped-release ratios under Super Loop. Each configuration specifies the task-set utilization U and the number of tasks.

Chunked EDF

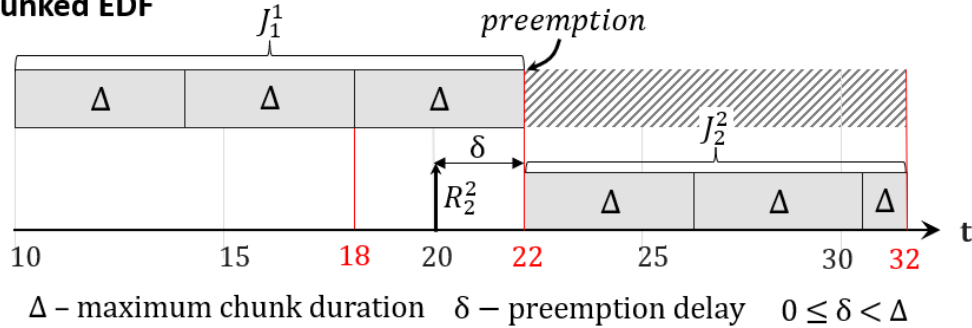


Fig. 4. Chunk-bounded preemption in the EDF implementation.

Algorithm 2 Chunked EDF Scheduler

```
1: function CHUNKEDEDF( $\mathcal{T}, \Delta$ )
2:   for all  $\tau_i \in \mathcal{T}$  do
3:      $t_i^{next} \leftarrow 0$ 
4:      $active_i \leftarrow false$ 
5:   end for
6:   while true do
7:      $t^{cur} \leftarrow \text{NOW}$ 
8:     for all  $\tau_i \in \mathcal{T}$  do
9:       if  $\neg active_i \wedge t^{cur} \geq t_i^{next}$  then
10:         $j \leftarrow 1 + t_i^{next} / T_i$ 
11:         $R_i^j \leftarrow t_i^{next}$ 
12:         $d_i^j \leftarrow R_i^j + D_i$ 
13:         $active_i \leftarrow true$ 
14:      end if
15:    end for
16:     $J_i^j \leftarrow \arg \min_{J_k^l \in \mathcal{R}} d_k^l \quad \triangleright \text{Earliest deadline}$ 
17:    if  $J_i^j \neq \emptyset$  then
18:      EXECUTECHUNK( $J_i^j, \Delta$ )
19:      if COMPLETED( $J_i^j$ ) then
20:         $f_i^j \leftarrow \text{NOW}$ 
21:        if  $f_i^j > d_i^j$  then
22:          RECORDMISS( $J_i^j$ )
23:        end if
24:         $t_i^{next} \leftarrow t_i^{next} + T_i$ 
25:        while  $t_i^{next} < f_i^j$  do
26:          RECORDSKIP( $\tau_i, t_i^{next}$ )
27:           $t_i^{next} \leftarrow t_i^{next} + T_i$ 
28:        end while
29:         $active_i \leftarrow false$ 
30:      end if
31:    end if
32:  end while
33: end function
```
